Chapter 4

Considering Additional Packages and Libraries You Might Want

IN THIS CHAPTER

The terms *package* and *library* refer to code written not by the original developers but rather by a third party. Such code doesn't necessarily appear as part of a language's default installation; in fact, none of the packages and libraries in this chapter comes along with a default installation. You must obtain and install each one in some way to use it.

Third-party code alleviates the need for you to write something yourself. In most cases, you find that such code contains essential features in an easy-to-use and consistent manner. The code's developers perform required updates and ensure that the code is as bug free as possible for you. You may find that the code is compiled in binary form or provided as source that is interpreted with the rest of your application. Some packages and libraries cost money to use, but none of the packages in this chapter cost anything unless you obtain a special

high-end version that provides some level of additional support or functionality.

REMEMBER The terms *package* and *library* prove confusing for many people, which isn't surprising. Different languages and different developers use the terms in various ways. For the purposes of this book, a *Python package* is equivalent to an *R library* in that it contains a collection of functions you download as a group.

Packages and libraries consist of smaller units to make working with the package or library easier and to reduce resource usage. For the purposes of this book, Python packages contain individual *modules* of like functionality. You work with a module to obtain a specific level of functionality after obtaining a package containing the module. Likewise, R libraries contain *packages*. The nuances among all these various packaging methods won't matter for this book.

With these naming differences in mind, this chapter begins by discussing how you use third-party code in a generic way to perform data science tasks. If you were to write a data science application completely from scratch, without the use of third-party packages or libraries, it would take an incredibly long time. However, you need to know that you could do it should you want to do so. Packages and libraries aren't magic — they're simply code.

The last two parts of the chapter discuss specific packages and libraries for each of the book's languages: Python and R. You use some of these packages and libraries in the book, but others answer specific needs that you won't have until you have spent more time working with data science. In fact, you may not use all the packages and libraries listed in this chapter, even if you learn how to perform data science tasks using all three languages.

Considering the Uses for Third-Party Code

As with languages, third-party code exists to fulfill the needs of the user. A computer understands only machine language — the essential language used to flip the switches within its processor. As with any code, an interpreter or compiler serves to translate the third-party code into machine language. Consequently, the computer doesn't care whether you use one scientific package or another. So your first concern is to choose a package or library that makes sense to you — one that makes tasks easier and reduces the amount of code you must write.

REMEMBER Developers who write third-party code used in packages and libraries are just like you: They possess various skill levels and sometimes make mistakes. Consequently, when you do find a package or library that you like, you need to consider how the developer wrote its code. You should consider these questions:

- Is the code relatively error free? (The bug-free anything is a myth.)
- Does the code enjoy community support?
- Will the developer (or development team) continue to support the code?
- Do independent benchmark tests exist that show that the code executes quickly?
- Has anyone validated the code's resource usage levels?
- Have tests shown correct outputs even when working with less than perfect inputs?

Even if you happen to find a high-quality package or library that has a superior feature set, you must still use it correctly. If you get a science package that specializes in data analysis, the lesser elements of the package may not perform well when attempting a machine learning task. Sometimes you must mix packages and libraries together in a smart manner to obtain a desired result for your particular application. The package or library developer has no way to know anything

about your application in advance, so the burden of choosing the correct features falls on you, which is where experimentation comes into play.

TIP The smart developer doesn't rely on just one or two packages or libraries. It's akin to creating a perfume. Using just one flower is nice, but it's hardly going to produce a spectacular result. Instead, the perfumer visits the garden and chooses just the right flowers with which to create the exquisite oils used in a perfume that dazzles. Likewise, a masterful developer will have an entire garden of packages and libraries from which to choose, mixing those features that specifically answer a particular application's needs.

Another important consideration is that many packages and libraries provide the means for addressing the disparity between language features. For example, you find that some Python packages help Python to address the statistical analysis disparity between Python and R. A package or library can address the concern where using a particular language almost meets a particular application need, but not quite.

Obtaining Useful Python Packages

Many developers use Python as a language of choice for general analysis needs in scenarios in which an application needs to perform a wide range of data science tasks rather than specialize in a particular area. However, Python isn't a general-purpose language like Java. You wouldn't use it to create a user interface for a refrigerator, even if that refrigerator is smart. Python's popularity stems from having just enough specialization to make it easy for someone who has to perform detailed analysis to learn, but it's not so specialized as to make some general tasks undoable. With this level of specialization in mind, the following sections discuss various Python packages used for certain kinds of analysis, but you should know that Python can do more. You

can find a significantly longer list of packages at `https://wiki.python.org/moin/UsefulModules`.

Accessing scientific tools using SciPy

The SciPy stack (`http://www.scipy.org/`) contains a host of other packages that you can also download separately. These packages provide support for mathematics, science, and engineering. When you obtain SciPy, you get a set of packages designed to work together to create applications of various sorts. These packages are

- NumPy
- SciPy
- matplotlib
- Jupyter
- SymPy
- pandas

Of course, these are just the packages listed on the SciPy main page. If you dig further into the details found at `https://www.scipy.org/about.html`, you discover other packages, called the SciPy ecosystem, that are built upon or around SciPy. For example, when it comes to data and computation, you find these packages:

- Scikit-image
- Scikit-learn
- h5py
- PyTables

The SciPy package itself focuses on numerical routines, such as routines for numerical integration and optimization. SciPy is a general-purpose package that provides functionality for multiple problem domains. It also provides support for domain-specific packages, such as Scikit-learn, Scikit-image, and statsmodels.

R intrinsically provides much of the SciPy functionality as part of the language. You can find discussions comparing R to SciPy functionality online. For example, you can find a comparison of R optim to SciPy optimize at `https://stackoverflow.com/questions/51813317/r-optim-vs-scipy-optimize`. Even though the functionality isn't precisely the same, you find that R and Python with SciPy can produce equivalent results.

Performing fundamental scientific computing using NumPy

The NumPy library (`https://numpy.org/`) provides the means for performing n-dimensional array manipulation, which is critical for data science work. NumPy functions also include support for linear algebra, Fourier transform, and random-number generation (see the listing of functions at `https://docs.scipy.org/doc/numpy/reference/routines.html`).

Many discussions online talk about how NumPy helps Python developers bridge the statistical processing gap with R (see `https://stackoverflow.com/questions/3545057/numpy-for-r-user` as an example). You also find that pandas is important in helping bridge this gap. However, when you use a package to overcome an apparent language deficiency, the only true way to determine which language is better is to perform tests. In addition, it's important to understand that you won't find a one-for-one correlation between the features that a package provides and the deficiencies that it helps to overcome. Complicating matters further, some R libraries, such as reticulate (see the "Using your Python code in R with reticulate" section,

later in this chapter, for details), let you load certain Python packages within R (see `https://cran.r-project.org/web/packages/reticulate/vignettes/python_packages.html` for details). Consequently, even if a Python package offers advantages over another language, the other language may still be able to use the Python package.

Performing data analysis using pandas

The pandas library (`http://pandas.pydata.org/`) provides support for data structures and data analysis tools. The library is optimized to perform data science tasks especially fast and efficiently.

The basic principle behind pandas is to provide data analysis and modeling support for Python that is similar to other languages, such as R. The focus of this modeling is the `dataframe` . You can find a comparison of pandas and R functionality on the site at `https://pandas.pydata.org/pandas-docs/stable/getting_started/comparison/comparison_with_r.html` (complete with some coding examples).

Implementing machine learning using Scikit-learn

The Scikit-learn library (`http://scikit-learn.org/stable/`) is one of a number of Scikit libraries that build on the capabilities provided by NumPy and SciPy to allow Python developers to perform domain-specific tasks. In this case, the library focuses on data mining and data analysis. It provides access to the following sorts of functionality:

- Classification
- Regression
- Clustering
- Dimensionality reduction
- Model selection
- Preprocessing

A number of these functions appear as chapter headings in the book. As a result, you can assume that Scikit-learn is the most important library for the book (even though it relies on other libraries to perform its work).

R doesn't offer a single library that precisely matches Scikit-learn. However, it does provide the caret library, which gives you similar functionality. You can find a comparison of the two at `https://www.analyticsvidhya.com/blog/2016/12/cheatsheet-scikit-learn-caret-package-for-python-r-respectively/`. The "Conducting advanced training using caret" section, later in this chapter, provides additional information. Another popular R library for machine learning is Mlr (`https://cran.r-project.org/web/packages/mlr/index.html`). Again, it offers some of what you find in Scikit-learn, but the two aren't a complete match (see the "Performing machine learning tasks using Mlr" section, later in this chapter, for more details).

TIP If you need a single-source solution for machine learning needs and don't mind jumping through a few hoops to get it, H2O (`https://www.h2o.ai/products/h2o/`) provides a solution for R, Python, and Scala developers.

Going for deep learning with Keras and TensorFlow

Keras (`https://keras.io/`) is an application programming interface (API) that is used to train deep learning models. An *API* often specifies a model for doing something, but it doesn't provide an implementation. Consequently, you need an implementation of Keras to perform useful work, which is where TensorFlow (`https://www.tensorflow.org/`) comes into play.

TIP You can also use Microsoft's Cognitive Toolkit, CNTK (`https://www.microsoft.com/en-us/cognitive-toolkit/`), or Theano (`https://github.com/Theano`) to implement Keras, but this book focuses on TensorFlow. A good reason to focus on TensorFlow is that you can find a version of it for R (`https://tensorflow.rstudio.com/`). Even though you find language differences in the three implementations, by using a single product, you significantly reduce your learning curve.

When working with an API, you're looking for ways to simplify your approach to tasks. Keras makes things easy in the following ways:

- **Consistent interface:** The Keras interface is optimized for common use cases with an emphasis on actionable feedback for fixing user errors.
- **Lego approach:** Using a black-box approach makes it easy to create models by connecting configurable building blocks together with only a few restrictions on how you can connect them.
- **Extendable:** You can easily add custom building blocks to express new ideas for research that include new layers, loss functions, and models.
- **Parallel processing:** To run applications fast today, you need good parallel processing support. Keras runs on both CPUs and GPUs.
- **Direct Python support:** You don't have to do anything special to make the TensorFlow implementation of Keras work with Python, which can be a major stumbling block when working with other sorts of APIs.

Plotting the data using matplotlib

The matplotlib library (`http://matplotlib.org/`) gives you a MATLAB-like interface for creating data presentations of the analysis you perform. The library is currently limited to 2-D output, but it still provides you with the means to express graphically the data patterns you see in the data you analyze. Without this library, you couldn't create output that people outside the data science community could easily understand.

R doesn't provide a matplotlib equivalent. Yes, you can still output plots, but Python appears to have a clear advantage in this case. The best libraries to use with R for output are ggplot2 (see the "Visualizing data using ggplot2" section of the chapter for details) and Esquisse (see the "Enhancing ggplot2 using esquisse" section of the chapter for details).

When working with R, you can also use reticulate to import matplotlib into your R configuration. The article at `https://rstudio.github.io/reticulate/articles/r_markdown.html` shows you how to perform this task. However, the implementation is less than perfect; you may need to use workarounds, as described in the article at `https://community.rstudio.com/t/matplotlib-in-line-plots-with-reticulate-on-rstudio-server/16357`.

Creating graphs with NetworkX

To properly study the relationships between complex data in a networked system (such as that used by your GPS setup to discover routes through city streets), you need a library to create, manipulate, and study the structure of network data in various ways. In addition, the library must provide the means to output the resulting analysis in a form that humans understand, such as graphical data. NetworkX (`https://networkx.github.io/`) enables you to perform this sort of analysis. The advantage of NetworkX is that nodes can be anything (including images) and edges can hold arbitrary data. These features allow you to perform a much broader range of analysis with NetworkX than using custom code would (and such code would be time consuming to create).

Even though the preferred package for Python is NetworkX, some developers use igraph (`https://igraph.org/redirect.html`) because it supports both R and Python (`https://igraph.org/python/`) directly. If you already have some code that relies on NetworkX, the discussion at `https://stackoverflow.com/questions/23235964/interface-between-networkx-and-igraph` tells you have to interface one with the other. The "Creating graphs with igraph" section of the chapter discusses the igraph in more detail.

Parsing HTML documents using Beautiful Soup

The Beautiful Soup library (`http://www.crummy.com/software/BeautifulSoup/`) download is actually found at `https://pypi.python.org/pypi/beautifulsoup4/4.3.2` . This library provides the means for parsing HTML or XML data in a manner that Python understands. It allows you to work with tree-based data.

Besides giving you a means for working with tree-based data, Beautiful Soup takes a lot of the work out of working with HTML documents. For example, it automatically converts the *encoding* (the manner in which characters are stored in a document) of HTML documents from UTF-8 to Unicode. A Python developer would normally need to worry about things like encoding, but with Beautiful Soup, you can focus on your code instead.

R uses the rvest (`https://cran.r-project.org/web/packages/rvest/index.html`) library to obtain similar (but not precisely the same) parsing results as Beautiful Soup. The

article at `https://www.dataquest.io/blog/python-vs-r/` gives an objective comparison of R and Python, including a segment on HTML document parsing. Even using rvest, R requires more code, which says a lot because R is normally quite frugal when it comes to code. The "Parsing HTML documents using rvest" section, later in this chapter, provides more insights into using rvest.

Locating Useful R Libraries

R supplies a wealth of built-in functionality for its core proficiency of statistical analysis, so you frequently find that you don't need to use libraries with it as often as if you were to use Python. However, you do need libraries at times, especially if you have certain goals that fall outside the normal range of R proficiencies. The following sections discuss some R libraries that you might want to add to your collection to perform data science tasks with greater ease. Be sure to also read through the "Obtaining Useful Python Packages" section, earlier in this chapter, if you want to understand how R and Python packages compare.

Using your Python code in R with reticulate

If you already work with Python and have a substantial investment in Python code, you can continue to use that investment in many cases by adding reticulate (`https://rstudio.github.io/reticulate/`) to your R toolbox. You use reticulate to access your Python code from R in a nearly seamless manner. In fact, you have access to four techniques for accomplishing this task:

- R Markdown
- Sourcing Python scripts
- Importing Python modules
- Using Python interactively within an R session

REMEMBER The reticulate library automatically marshals your data between languages. For example, it can translate between R and pandas `dataframes`, among other objects. In addition, you gain access to your Python environment, such as the one you configure in Chapter 3 of this minibook.

TIP There are limits to what the reticulate library can do for you. For example, it won't suddenly force R to allow passing of data by reference rather than value. So, you can't use it to solve certain R graphics plotting issues. However, you can find it invaluable in overcoming other R deficiencies, such as performing some types of data fitting tasks. The downloadable source (as described in the Introduction) contains examples of using reticulate to overcome learning issues discussed in Book 4, Chapter 3; Book 5, Chapter 2; and Book 5, Chapter 5. In addition, you find it called in as a separate library in Book 5, Chapter 3 and Book 6, Chapter 3.

Conducting advanced training using caret

The Classification and Regression Training (CARET) library (normally written as *caret*) (`http://topepo.github.io/caret/index.html`) originally started as a method to provide consistent access to the built-in R functions. It does more than simply provide access now; you use this library to perform the following:

- Visualizations
- Data splitting
- Preprocessing
- Feature selection
- Model tuning using resampling
- Variable importance estimation

TIP The documentation provided on the host site is extensive but can be a little hard to follow, and it doesn't always contain the complete code needed for an example to work. The Cran site at `https://cran.r-project.org/web/packages/caret/vignettes/caret.html` provides additional information that makes using this library easier.

Performing machine learning tasks using mlr

Machine Learning in R (MLR) (`https://mlr.mlr-org.com/`) standardizes the interface provided for the built-in R machine learning functions, making them significantly easier to use. In addition, you write less code when you use the mlr library because it consolidates some functionality. The site page offers a host of features that this library supports, but here is a quick overview of the supervised and unsupervised learning functionality:

- Classification
- Regression
- Survival analysis
- Evaluation and optimization methods
- Clustering

Visualizing data using ggplot2

When working declaratively, you tell a library what you want done, but you let the library decide how to do it. The ggplot2 library (`https://ggplot2.tidyverse.org/`) relies on the Grammar of Graphics (GG) system to declare how to present information onscreen so that others can easily understand it. As with matplotlib, you might have to put in some effort to discover precisely how to perform every task in ggplot2, which is why the site page tells you about places to find tutorials. However, one of the more helpful aids with this library

is the cheat sheet found at `https://github.com/rstudio/cheatsheets/blob/master/data-visualization-2.1.pdf`.

Enhancing ggplot2 using esquisse

When working with complex products that help you display data in graphic form, trying to get started can prove difficult. The ggplot2 library gives you significant flexibility, but the learning curve can be steep. The esquisse add-on (`https://github.com/dreamRs/esquisse`), which isn't a library, reduces the complexity of using ggplot2 by helping you create a starting point interactively.

REMEMBER You use a GUI to design the output you want to present, but in a simple manner. The add-on doesn't provide access to advanced ggplot2 features — it focuses on making things simple so that you can see what to do at the outset and then add to the result you get to obtain the refinements you need.

Creating graphs with igraph

Network graphs help you present complex data in a visual manner. For example, you might want to create an application that shows how to get from one place in a city to another. To create such a presentation, you need a network graph. The igraph library (`https://igraph.org/redirect.html`) enables you to add network graphing functionality directly to both Python and R. (The R-specific details appear at `https://igraph.org/r/`.) This library focuses on efficiency, portability, and ease of use, so you might find that it doesn't contain all the functionality offered by NetworkX (described earlier in the chapter). You use igraph to

- Generate graphs

- Compute centrality measures
- Compute path length based properties

Parsing HTML documents using rvest

HTML documents contain all sorts of formatting in the form of tags that make working with the data nearly impossible in any significant way without parsing. The rvest library (`https://github.com/tidyverse/rvest`) focuses on performing simple forms of parsing for elements such as HTML tables, as described in the article at `https://blog.rstudio.com/2014/11/24/rvest-easy-web-scraping-with-r/`. The point is to get the data into a form that you can use to create a `dataframe` for additional processing.

TIP Getting started with rvest may take a little time because of the complexity of formatting used by most web pages. The tutorial at `https://www.analyticsvidhya.com/blog/2017/03/beginners-guide-on-web-scraping-in-r-using-rvest-with-hands-on-knowledge/` offers a quick hands-on tutorial that makes working with rvest easier.

Wrangling dates using lubridate

To obtain correct analysis output in most cases, you need to deal with dates and times. However, dates and times come in many forms, so interpreting them can prove problematic. For example, you must ask yourself when looking at 02/03/19 whether the date represents 3 February 2019 or 2 March 2019. In fact, it could represent something completely different, such as 19 March 2002. You just don't know unless you have some means for interpreting the date, such as through lubridate (`https://lubridate.tidyverse.org/`). By knowing the point of origin for the date, you can interpret it correctly.

Times can be even harder. Now you must also consider issues other than simply format. Two times might be in the same format, but reflect different time zones.

You must also consider the issue of dissection. For example, you might need to know what day of the week 3 February 2019 happened on. Unfortunately, the information doesn't appear as part of the date; you must dissect the date and then use the information to look up the day of the week. You can also use lubridate to perform date and time math to determine things like intervals.

Making big data simpler using dplyr and purrr

Often, a language comes with functionality that helps you perform a wealth of useful tasks, but accessing that functionality can prove difficult. R enables you to manage huge datasets in various ways so that you can make hard problems simple. However, accessing that functionality can prove difficult, and a library like dplyr (`https://dplyr.tidyverse.org/`) reduces your workload. Using dplyr, you can

- Fit a specific model to subsets of a data frame
- Calculate summary statistics for each data group
- Perform group-wise transformations, such scaling or standardizing

The way in which dplyr performs its task is to

- Make calling conventions more consistent
- Use the `foreach` package to make parallelism easier to manage
- Improve the input and output functionality for `dataframes`
- Monitor processes with enhanced error handling

Unfortunately, dplyr works only with `dataframes`. When working with lists, you want to use purrr (`https://purrr.tidyverse.org/`) instead. When working with purrr, you rely on functional programming techniques to map data in various ways, such as by splitting large

pieces into small ones. The first argument for all purrr functions is the data, so this library makes working with pipes incredibly easy.

Table of contents collapsed